Continuous Record/Replay to SSD (v2 driver + apps)

This describes the `v2` continuous (gap-free) record/replay stack — `app_sw/.../dma/v2/driver/t510_dma_stream.c` (kernel) and `app_sw/.../dma/v2/app/{rx_stream,tx_stream}.c` (user space) — and the reasoning behind its design. For the throughput ceiling this design is built against (RFDC rate vs. NVMe link), see `docs/OPEN_QUESTIONS.md` §1.6/§4.4.

It supersedes `v1` (`t510_dma_loopback.c` + `rx.c`/`tx.c`), which is DDR-loopback-only and stops/restarts the DMA channel between every 2 MB block (a real gap between blocks — see `v1`'s known issues in `docs/sw-arch.md`).

---

1. Kernel driver (`t510_dma_stream.c`)

1.1 The core problem it solves

`v1` ran RX as repeated one-shot transfers: `IOC_START` → DMA fills one buffer → `IOC_STOP` (`dmaengine_terminate_sync` + a mandatory 20 ms settle) → `IOC_START` again for the next buffer. Each stop/restart cycle is a real gap in the sample stream — acceptable for a loopback demo, not for GNSS record/replay where every sample's timing matters.

1.2 Design choice: one cyclic descriptor, forever

`t510_dma_v2_start_rx_locked()` / `_start_tx_locked()` each call `dmaengine_prep_dma_cyclic()` **once**, over the *entire* ring (`ring_bytes = period_bytes * num_periods`), and submit it with `dma_async_issue_pending()`. The AXI DMA engine then loops period 0,1,2,...,N-1,0,1,2,... **indefinitely** with no CPU intervention and no terminate/restart between periods — only `STOP_RX`/`STOP_TX` (`dmaengine_terminate_sync`) ends the chain, once per session.

This shifts the synchronization problem from "stop hardware, copy, restart hardware" to "let hardware run continuously, and have user space race the hardware's period pointer" — i.e. a classic lock-free ring buffer between one producer (DMA) and one consumer (user space), or vice versa for TX.

1.3 Design choice: separate RX and TX rings, separate channels

`tdev->rx_buf`/`rx_chan` and `tdev->tx_buf`/`tx_chan` are completely independent — separate `dmam_alloc_coherent()` allocations, separate AXI DMA channels (`dma_request_chan(..., "rx")` / `"tx"`), separate atomic counters (`rx_hw_periods`/`tx_hw_periods`) and wait queues. The only shared state is `tdev->lock` (a mutex guarding START/STOP/channel request-release).

This is what lets `rx_stream` and `tx_stream` run as two independent processes simultaneously (live capture + replay-under-test at the same time), each only touching its own direction.

1.4 Design choice: `dmam_alloc_coherent` + `mmap`, not `read()`/`write()`

The rings are allocated once at probe time with `dmam_alloc_coherent()` (physically contiguous, CPU-uncached, DMA-coherent memory) and exposed to user space via `dma_mmap_coherent()` in `t510_dma_v2_mmap()`:

- offset `0` → RX ring, `PROT_READ` (user space only ever reads DMA output)
- offset `ring_bytes` → TX ring, `PROT_READ | PROT_WRITE` (user space fills, hardware reads)

Because the memory is DMA-coherent and non-cacheable, **no `dma_sync_*`/cache-maintenance calls are needed** on either side — what the CPU writes is immediately visible to the DMA engine and vice versa. This also means **zero-copy**: the only data movement is DMA-engine ↔ DDR and `fread`/`fwrite` ↔ DDR; there is no separate "DMA

buffer → app buffer" copy.

The tradeoff is the contiguous-allocation requirement: `ring_bytes` must fit in one CMA block per ring (`2 * num_periods * period_bytes` total, contiguous *per ring*), which bounds how large the rings can be without increasing the kernel's CMA pool (`cma=` bootarg) — see `docs/OPEN_QUESTIONS.md` §4.4 / architecture review notes.

1.5 Design choice: counters + `wait_event_interruptible_timeout`, not a full producer/consumer queue

The driver exposes the *minimum* state user space needs:

- `rx_hw_periods` / `tx_hw_periods`: monotonically increasing counts of periods the hardware has completed, incremented from the DMA completion callback (`t510_dma_v2_rx_callback`/`_tx_callback`) — this runs in interrupt/tasklet context, so it just does an `atomic64_inc` and `wake_up_interruptible`, nothing blocking.
- `WAIT_RX`/`WAIT_TX`: blocks the calling thread until the corresponding counter changes (or ~1s elapses, or the direction is stopped), via `wait_event_interruptible_timeout`. The 1s timeout means user space's main loop periodically re-checks `g_stop` (signal handling) even if the hardware were to stall completely.

There is deliberately **no kernel-side bookkeeping of how far user space has progressed** (`local_count` is purely a user-space variable). The driver's job is only "tell me when a period completes"; *how far behind is safe* (the `num_periods - 1` margin discussed in §3.5) is entirely a user-space policy decision, kept out of the kernel.

1.6 Design choice: `release()` is a deliberate no-op

`t510_dma_v2_release()` does nothing. If it called `STOP_RX`/`STOP_TX` on `close()`, then `rx_stream` exiting would also kill an independently-running `tx_stream`'s playback (both processes `open()` the same device). Instead, each tool calls its own `STOP_RX`/`STOP_TX` explicitly before exiting, and `t510_dma_v2_remove()` (module unload) calls `t510_dma_v2_stop_all_locked()` as the final safety net.

1.7 Why it binds to the same device-tree node as `t510_dma_loopback`

`t510_dma_v2_of_match` matches both `"antsdr,t510-dma-loopback"` (the existing DT node, unchanged) and a new `"antsdr,t510-dma-stream"` string — so no device-tree/bitstream changes are required to switch drivers, just `rmmod t510_dma_loopback; insmod t510_dma_stream.ko`. Only one driver can be bound to the node at a time, which is enforced by the platform-driver model itself (no extra code needed).

1.8 Module parameters: `period_bytes` / `num_periods`

Both are `module_param(..., 0444)` (read-only after load, i.e. set via `insmod foo=bar`, not changeable at runtime). Defaults (`T510_DMA_V2_PERIOD_BYTES` = 1 MiB, `T510_DMA_V2_NUM_PERIODS` = 16, so 16 MiB/ring, 32 MiB total) are deliberately conservative — large enough to absorb a few ms of SSD jitter, small enough to allocate without touching `cma=`. `period_bytes` must be a `PAGE_SIZE` multiple (required by `dma_mmap_coherent`); both are validated in `t510_dma_v2_probe()`. Increasing either is a pure sizing/tuning change — no other code depends on the defaults (`rx_stream`/`tx_stream` always read actual sizes from `GET_STATUS`).

---

2. App: `rx_stream` (capture → SSD)

2.1 Core loop

```
START_RX loop: GET_STATUS -> rx_hw_periods (hw_count) if hw_count == local_count: WAIT_RX, continue # nothing new yet if hw_count - local_count >= num_periods: ... # overrun, see §2.2 while local_count < hw_count: idx = local_count % num_periods write period[idx] to --output (or discard/dump, see §3) local_count++ STOP_RX
```

2.2 Design choice: explicit overrun detection with resync, not "best effort"

If `rx_stream` is ever `>= num_periods` periods behind, the oldest unread period(s) have already been overwritten by the cyclic DMA — that data is permanently gone. Rather than silently reading garbage or crashing, the code:

1. Computes exactly how many periods were lost: `lost = (hw_count - local_count) - (num_periods - 1)`.
2. Logs it and adds it to `overrun_count` (written to `--meta`).
3. Resyncs `local_count = hw_count - (num_periods - 1)` — the oldest period that is *guaranteed* complete and not concurrently being written (the period at index `hw_count % num_periods` is the one the DMA is writing *right now*; touching it would read a torn period without it being counted as loss).

This makes data loss **visible and quantified** in the metadata rather than either (a) producing a corrupt file with no indication, or (b) the tool aborting outright — for a debugging/bring-up tool, "keep going and tell me exactly what you lost" is more useful than either extreme.

2.3 Design choice: blocking `WAIT_RX` instead of polling

`WAIT_RX` blocks (up to ~1s) until `rx_hw_periods` changes. This avoids a busy-poll loop burning CPU while waiting for the next ~1 MiB period (at ~983 MB/s, periods complete roughly every ~1 ms with the 1 MiB default — frequent enough that polling would be wasteful, infrequent enough that blocking is free). The 1s timeout is a safety net so `g_stop` (Ctrl+C) and `--duration` are still checked even if the hardware somehow stalls.

2.4 Design choice: `fopen`/`fwrite` with a 1 MiB stdio buffer, not raw `write()`

The RX ring is `mmap`'d `PROT_READ` and copied out with buffered stdio (`setvbuf(out_fp, NULL, _IOFBF, 1U << 20)`). This is the simplest correct implementation and was judged "good enough" because the DMA path itself is already the bottleneck-free part of the pipeline (see architecture review in `docs/OPEN_QUESTIONS.md` §4.4) — the **NVMe link**, not this extra CPU copy, is the binding constraint. If profiling later shows the stdio copy + page-cache interaction matters, the documented next step is `O_DIRECT` + raw `read()`/`write()` with `period_bytes`-aligned I/O — not yet implemented, kept as a known optimization rather than premature complexity.

2.5 `--sample-rate` / `--lo-freq` / `--adc-bits`: metadata only

These do **not** configure the RFDC — that remains the job of `t510_rf_init` (run twice, per the top-level bring-up sequence). They are recorded into `--meta` purely so a `.bin` capture file is self-describing for later analysis (what frequency/rate it was captured at). `--adc-bits` is a placeholder for a future bit-packing stage; samples are currently always raw `int16` regardless of this value.

2.6 Shutdown and `--meta`

On `SIGINT`/`SIGTERM`, the in-flight inner `while` loop finishes the period it's currently writing, then the outer loop exits, `STOP_RX` is issued (`dmaengine_terminate_sync` — may truncate the period the DMA was *actively filling*, a one-time end-of-session truncation of at most one period), files are flushed/closed, and `--meta` is written with final counts (`periods_written`, `bytes_written`, `overrun_count`, flush-related fields, `elapsed_sec`, `status`). `--meta` is the single source of truth for "did this capture lose anything" — every loss mechanism (overrun, flush-discard, stop-truncation) is accounted for there.

---

3. App: flush/dump (`--flush-periods` / `--dump-flush`)

3.1 Problem: stale FIFO backlog at start of capture

Between `STOP_RX` (or module load) and the next `START_RX`, the PL's `axis_data_fifo_2` can keep accumulating samples from the still-running RFDC (the FIFO sits *upstream* of the DMA in the dataflow — see `docs/rfdc-to-ddr-dataflow.md`). When `START_RX` issues the cyclic descriptor, the **first period(s) read out may be that pre-existing backlog**, not samples captured from the moment `START_RX` was called. `v1` worked around this with a hardcoded `T510_DMA_RX_FLUSH_PERIODS = 3` one-shot discard transfers before every real capture.

3.2 Design choice: opt-in, app-side, configurable count

`v2` does not replicate `v1`'s hardcoded flush in the driver. Instead, `rx_stream --flush-periods N` (default `0` = disabled) discards the first `N` periods **in user space**, after `START_RX`:

- Each discarded period is still read off the ring (`local_count++`), so the ring/`local_count` bookkeeping — and overrun detection — stays correct; the driver/hardware are completely unaware flushing is happening.
- Discarded periods are simply not written to `--output`.

Making this opt-in and app-side means: (a) it costs nothing when not needed (`N=0`, the common case once a board's FIFO-fill behavior is well understood), (b) `N` can be tuned per-situation instead of a fixed `3`, and (c) it requires no driver/ABI changes — purely a consumer-side policy, which matches the driver's "just report period completions" philosophy (§1.5).

3.3 Design choice: `--dump-flush PATH` for inspection rather than blind discard

For bring-up/debugging, simply throwing away the flushed periods makes it impossible to verify *what* was actually in the FIFO at start (was it really stale data? how much? does it look like valid IQ?). `--dump-flush PATH` writes the discarded periods to a **separate file** instead of dropping them — same `write_full()` helper, same 1 MiB buffered `fopen`, opened and error-checked alongside `--output` so a failure here aborts the run exactly like a failure on the main output (no silent partial state).

This was added specifically to support the kind of manual "capture-and-inspect-the-first-buffer" debugging done during earlier bring-up sessions, without needing a one-off hacked-up build of `rx_stream` each time.

3.4 `--meta` accounting for flush

`flush_periods_requested` and `flush_periods_done` are both recorded (they can differ if `--duration` expires while still flushing, or on early abort/Ctrl+C during the flush phase), plus `dump_flush_file` (empty string if `--dump-flush` wasn't given). `periods_written`/`bytes_written` count only the *non-flushed* periods that went to `--output` — `local_count - (flush_periods - flush_remaining)` subtracts out whatever was flushed, so `--meta` always reflects the real content of `--output` regardless of flush settings.

---

4. App: `tx_stream` (SSD → replay)

4.1 Core loop — mirror image of `rx_stream`

```
pre-fill all num_periods periods of the TX ring from --input START_TX loop: GET_STATUS -> tx_hw_periods (hw_count) if local_count <= hw_count: ... # underrun, see §4.2 if local_count >= hw_count + num_periods: WAIT_TX, continue # ring full refill period[local_count % num_periods] from --input local_count++ STOP_TX
```

4.2 Design choice: pre-fill before `START_TX`

Unlike RX (where the ring starts empty and hardware fills it), TX hardware starts *playing* immediately on `START_TX` — if the ring weren't pre-filled, the first `num_periods` periods played would be whatever garbage/zeros were in the freshly-allocated buffer. `tx_stream` therefore fills all `num_periods` periods from `--input` **before** issuing `START_TX`, so

playback from time zero is real data. `dmam_alloc_coherent` buffers are zeroed at probe time, so even an immediate `STOP_TX` (or end-of-file with no `--loop`) plays defined silence (zeros), not uninitialized memory.

4.3 Design choice: underrun handling mirrors overrun, with a +1 asymmetry

If `local_count <= hw_count`, the hardware has played (or is currently playing, for `local_count == hw_count`) a period `tx_stream` hasn't refilled yet — stale data was/is being replayed. Symmetric to RX:

- `lost = (hw_count + 1) - local_count` periods counted as underrun.
- Resync to `local_count = hw_count + 1` — i.e. *skip past* the currently-playing period (refilling it now would tear it) and target the next one.

The `+1` (vs. RX's `-1`) reflects the different reference point: RX resyncs to the newest period guaranteed *not* currently being written; TX resyncs to the next period that will *not* be played before it can be refilled.

4.4 `--loop` vs. EOF handling

Without `--loop`, `fill_one_period()` zero-pads a short final read and sets `eof_reached`; the main loop then drains (`while local_count <= hw_count: wait`) until every pre-filled/refilled period has actually played, then exits — so a finite file plays exactly once, including its last (zero-padded) partial period, with no extra repeats or early cutoff. `--loop` instead `fseek`s back to the start of `--input` transparently inside `fill_one_period()`, so playback continues indefinitely until `--duration` expires or Ctrl+C.

4.5 Why TX has no flush/dump equivalent

Flush/dump (§3) addresses *stale data already in PL hardware* before a capture starts — a concept specific to RX's FIFO-then-DMA dataflow. TX has no analogous hardware-side backlog: the TX ring is explicitly pre-filled by `tx_stream` itself (§4.2) before `START_TX`, so there is nothing hardware-originated to discard or inspect at start.

---

5. Concurrent RX + TX

Because the driver gives RX and TX fully independent rings, channels, counters and wait queues (§1.3), and `release()` is a no-op (§1.6), `rx_stream` and `tx_stream` can run as two separate processes against the same `/dev/t510_dma_stream`, each driving only its own direction:

```
./rx_stream -o /mnt/ssd/capture.bin -t 300 -m capture.meta & ./tx_stream -i /mnt/ssd/replay.bin
-L -m replay.meta & wait
```

This supports the target use case directly: capture a live antenna feed while simultaneously replaying a previously-recorded signal into a receiver under test.